C++ Variable Types

A variable is a name which is associated with a value that can be changed. For example, when I write int value {38}; here variable name is value which is associated with value 33, int is a data type that represents that this variable can hold integer values. The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive.

Rules for Assigning Name to a Variable in C++

1. All variable names must begin with a letter of the alphabet or an. underscore (_).
2. After the first initial letter, variable names can also contain letters and numbers.
3. Uppercase characters are distinct from lowercase characters; this is because C++ is a case sensitive language. A variable with name hours is different with HOURS in C++
4. You cannot use a C++ keyword (reserved word) as a variable name.(e.g int, bool, char cannot be used as variable name in C++ but if you want to use one such as the int then you have to add a prefix such as int_score.
5. Variable names in C++ can range from 1 to 255 characters
6. No spaces or special characters are allowed.

Variable types in C++

In C++, there are different **types** of variables (defined with different keywords), for example:

- int - stores integers (whole numbers), without decimals, such as 258 or -258
- double - stores floating point numbers, with decimals, such as 27.99 or -27.99

- char - holds character value like 'c', 'F', 'B', 'p', 'q' etc.. Char values are surrounded by single quotes
- string - stores text, such as "Good Morning ! ". String values are surrounded by double quotes
- bool - stores values with two states: true or false

Each variable while declaration must be given a **datatype**, on which the memory assigned to the variable, depends

Syntax for variable declaration: You can declare a variable in C++ using the following syntax:

datatype <Variable name>

One Value to Multiple Variables

You can also assign the **same value** to multiple variables in one line:

Example

```
int x, y, z;
x = y = z = 50;
cout << x + y + z;
```

Displaying your Variable:

The **cout** object is used together with the **<<** operator to display variables. To combine both text and a variable, separate them with the **<<** operator:
Example:

int my_Score{}
Float School_fees{}
Boolean gender{}

Example 1.2

```
#include <iostream>
using namespace std;

int main() {
  int my_score {55};
  cout << "This display my score "<< my_score;
```

```
  return 0;
}
```
Output:
```
This display my score 55
```

Example 1.3 (to declare and display other variable)

```
#include <iostream>
using namespace std;

int main() {
  Char My_Grade{'A'};
  Float Average{38.5};
  bool gender{'F'};
  cout << "My Grade in this course would be "<< My_Grade;
  cout << "The Average Score of my CA is "<< Average;
  cout << "The Colleague next to me in the class is "<<gender;
  return 0;
}
```
Output:
```
My Grade in this course would be A
The Average Score of my CA is 38.5
The Colleague next to me in the class is F
```

To declare more than one variables of the same data type you use a comma to separate them e.g

```
Int x=25, y=33,z=24;
Cout<<x<<" "<<y<<" "<<z;
```

C++ Identifiers

We identify all C++ variables with a unique name, which are called identifier, but identifier can be written as a single character such x, r, and y. it is advisable to use descriptive name such as (age, sum, Average, fees, sum, total etc.). This would make your program to give meaning to who is reading or try to modify it in future.

Example 1.4

If you declare an identifier as:

int my_Budeget {50000} // this has a meaning to anyone reading it which is referring to a Budget amount; but if you name it as this

int x{50000}// this is correct syntactically but would have no meaning to the reader.

Note: ***The general rules for assigning a name to an identifier is the same the rules for assigning name to a variable in C++.***

C++ CONSTANTS

A constant is an identifier/variable whose value does not change or overwritten in the process of executing a program in C++. A valid constant name can start with a letter or an underscore. Numbers are allowed, but they cannot be used at the beginning of the constant name. For example, the constant name 1First_Name is not properly written. Instead, you might use something like First_Name1 or First1_Name.

Example 1.5

```
const int Emp_Tax {305};  // Emp_Tax will always be 305
Emp_Tax {25};  // error: assignment of read-only variable ' Emp_Tax '
```

if you properly code the program as follows:

```
#include <iostream>
using namespace std;

int main() {
  const int Emp_Tax{305};
  Emp_Tax{25};
  cout << Emp_Tax;
  return 0;
}

Output:
error: assignment of read-only variable 'Emp_Tax'
```

Assignment:

1. Use the correct keyword to get user input, stored in the variable **x**:

```
int x;
cout << "Type a number: ";
      >>      ;
```

2. Write a C++ Program that would declare the variables and its equivalent data type for the following supplied information:

a) ₦38,400
b) 25×10^5
c) True or False
d) Hello Dud!
e) 3.46

3. Fill in the missing parts to print the sum of two numbers (which is put in by the user):

```
int x, y;
int sum;
cout << "Type a number: ";
[ ] >> [ ];
cout << "Type another number: ";
[ ] >> [ ];
sum = x + y;
cout << "Sum is: " << [ ];
```

4. Add the correct data type for the following variables:

```
[ ] Num1 = 9;
[ ] DoubleNum= 8.99;
[ ] Grade = 'A';
[ ] myBool = false;
[ ] remark = "Passed A Course";
```

5. Fill in the missing parts to create three variables of the same type, using a **comma-separated list**:

```
[ ] x1 =3[ ] x2 = 2[ ] X3 = 5;
cout << x1 + x2 + x3;
```

6. Take two integer inputs from user. First calculate the sum of two then product of two. Finally, print the sum and product of both obtained results.

7. Create two boolean variables, named yay and nay, and add appropriate values to them:

```
[ ] [ ] = [ ];
[ ] [ ] = [ ];
```

8. Take value of length and breadth of a rectangle from user as float. Find its area and print it on screen after type casting it to int.

9. Write a program to assign a value of 100.235 to a double variable and then convert it to int.

10. What is **difference between Declaration and Definition of a variable.**

C++ Operators

Operators are used to perform operations on variables and values.

In the example below, we use the **+ operator** to add together two values:

```
int x = 100 + 50;
```

Although the **+** operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

Example

```
int sum1 = 52 + 25;         // 77 (52 + 25)
int sum2 = sum1 + 250;       // 102 (77 + 25)
int sum3 = sum2 + sum2;      // 179 (102 + 77)
```

C++ divides the operators into the following groups:

1) Arithmetic operators
2) Assignment operators
3) Comparison operators
4) Logical operators
5) Bitwise operators

Operator	*Name*	*Description*	*Example*
+	*Addition*	*Adds together two values*	*X+y*
-	*Subtraction*	*Subtracts one value from another*	*x-y*
***	*Multiplication*	*Multiplies two values*	*X*y*
/	*Division*	*Divides one value by another*	*x/y*
%	*Modulus*	*Returns the division remainder*	*X%y*
++	*Increment*	*Increases the value of a variable by 1*	*++x*
--	*Decrement*	*Decreases the value of a variable by 1*	*--y*

C++ Assignment Operators

Assignment operators are used to assign values to variables.

In the example below, we use the **assignment** operator (**=**) to assign the value **10** to a variable called **x**:

Example

int x = 10;

The **addition assignment** operator **(+=)** adds a value to a variable:

Example

int x = 10;

x += 5;➔x=x+5➔10+5

A list of all assignment operators:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison Operators

Comparison operators are used to compare two values (or variables). This is important in programming because it helps us to find answers and make decisions.

The return value of a comparison is either **1** or **0**, which means **true** (1) or **false** (0). These values are known as **Boolean values**, and you will learn more about them in the Booleans and If..Else in subsequent lecture

In the following example, we use the **greater than** operator (>) to find out if 5 is greater than 3:

Example

```
int x = 5;
int y = 3;
cout << (x > y); // returns 1 (true) because 5 is greater than 3
```

A list of all comparison operators:

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Example 1 (==)

```
#include <iostream>
using namespace std;
int main() {
  int x = 5;
  int y = 3;
  cout << (x == y); // returns 0 (false) because 5 is not equal to 3
  return 0;
}
```

Example 2 (!=)

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (x != y); // returns 1 (true) because 5 is not equal to 3
  return 0;
}
```

Example 3 (>)

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (x > y); // returns 1 (true) because 5 is greater than 3
  return 0;
}
```

Exercise Try For the other Comparison Operators

Logical Operators

As with comparison operators, you can also test for **true** (1) or **false** (0) values with **logical operators**.

Logical operators are used to determine the logic between variables or values:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Example 1 (&&)

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (x > 3 && x < 10); // returns true (1) because 5 is greater than 3 AND 5 is less than 10
  return 0;
}
```

Example 2 (||)

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (x > 3 || x < 4); // returns true (1) because one of the conditions are true (5 is greater than 3, but 5 is not less than 4)
  return 0;
}
```

Example 3 (!)

```
#include <iostream>
using namespace std;

int main() {
  int x = 5;
  int y = 3;
  cout << (!(x > 3 && x < 10)); // returns false (0) because ! (not) is used to reverse the result
  return 0;
}
```

C++ Strings

Strings are used for storing text.

A **string** variable contains a collection of characters surrounded by double quotes:

Example

Create a variable of type **string** and assign it a value:

```
string greeting = "Hello";
```

To use strings, you must include an additional header file in the source code, the **<string>** library:

Example

```
// Include the string library
#include <string>

// Create a string variable
string greeting = "Hello";
```

ASSIGNMENT

1. Fill in the missing part to create a **greeting** variable of type **string** and assign it the value **Hello**.

```
[    ] [    ] = [    ];
```

2. Use the **addition assignment** operator to add the value 5 to the variable x.

```
int x = 10;
x      5;
```

3. Use the correct operator to increase the value of the variable x by 1.

```
int x = 10;
      x;
```

String Length

To get the length of a string, use the **length()** function:

Example

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
cout << "The length of the txt string is: " << txt.length();
```

Tip: You might see some C++ programs that use the **size()** function to get the length of a string. This is just an alias of **length()**. It is completely up to you if you want to use **length()** or **size()**:

User Input Strings

It is possible to use the extraction operator **>>** on **cin** to display a string entered by a user:

Example

```
string firstName;
cout << "Type your first name: ";
cin >> firstName; // get user input from the keyboard
cout << "Your name is: " << firstName;
```

```
// Type your first name: John
// Your name is: John
```

However, `cin` considers a space (whitespace, tabs, etc) as a terminating character, which means that it can only display a single word (even if you type many words):

Example

```
string fullName;
cout << "Type your full name: ";
cin >> fullName;
cout << "Your name is: " << fullName;

// Type your full name: John Doe
// Your name is: John
```

From the example above, you would expect the program to print "John Doe", but it only prints "John".

That's why, when working with strings, we often use the `getline()` function to read a line of text. It takes `cin` as the first parameter, and the string variable as second:

Example

```
string fullName;
cout << "Type your full name: ";
getline (cin, fullName);
cout << "Your name is: " << fullName;

// Type your full name: John Doe
// Your name is: John Doe
```